

CherryPicker: Solver Description for Rooted Maximum Agreement Forest

Rishank Goyal ✉

The Adelaide University, Australia

Srinibas Swain ✉

The Adelaide University, Australia

Abstract

CherryPicker is the certified-oriented version of our PACE 2026 exact-track solver for rooted Maximum Agreement Forest on rooted binary trees. The submitted program is a single statically linked C++17 executable. It dispatches two-tree instances to a bundled RS [1]/HiGHS (an ILP solver) route and instances with at least three trees to a many-tree core that prints only after validation and an exact lower-target certificate. The many-tree build disables the guarded stable-exchange and empirical acceptance paths used by the aggressive version.

2012 ACM Subject Classification Theory of computation → Design and analysis of algorithms; Applied computing → Bioinformatics

Keywords and phrases maximum agreement forest, rooted phylogenetic trees, SAT, certified search

Category PACE 2026 Exact Track Solver Description

Supplementary Material <https://github.com/alienblackk/CherryPicker>

1 Model and Validation

The input trees $T_1, \dots, T_t$ share a taxon set X , $|X| = n$. An answer is a partition $\mathcal{F} = \{C_1, \dots, C_k\}$ of X . It is a rooted agreement forest if each restriction $T_j|_{C_i}$ has the same rooted topology for all trees j , and if the minimal connecting subtrees of different components are edge-disjoint in every input tree. The objective is to minimise k .

CherryPicker uses an independent validator before any forest can be printed. For each block it compares canonical rooted restriction signatures across all input trees. It also records all edges used by the block's minimal connecting subtree in every tree and rejects the candidate if two blocks share an edge. Finally, it checks that every taxon occurs exactly once.

► **Lemma 1.** *Every forest accepted by the validator is a feasible rooted agreement forest.*

Proof. The leaf checks prove that the blocks partition X . Matching restriction signatures prove that every block induces one rooted topology across all input trees. Pairwise disjoint recorded edge sets give the required edge-disjointness in every tree. These are exactly the feasibility conditions for rooted agreement forests. ◀

2 Many-Tree Certified Core

For number of input trees, $t \geq 3$, CherryPicker constructs validated incumbents using the same component generators as the aggressive solver: common subtree and cherry reductions, chain-derived components, anchored greedy merging, pair augmentation, high-divergence packing, local exchange, and bounded repacking. These steps only provide feasible upper bounds. In the CherryPicker build `ALLOW_STABLE_EXCHANGE_DEFAULT=0` and `ALLOW_EMPIRICAL_EXACT_DEFAULT=0`, so an incumbent is not printed unless a separate exact certificate closes the lower target.

The core also uses common-cluster recursion, following cluster partitioning in MAF computation [2] and marker semantics from cluster reductions [3]. If a leaf set C is a clade in every input tree, the solver creates inside and outside subinstances, both with and without a marker. Let B, T be the unmarked optimum counts and BM, TM the marker counts. If $BM = B$ and $TM = T$, the boundary component is joined and the combined count is $B + T - 1$; otherwise it is $B + T$. The recombined forest is validated on the original instance.

3 Exact Certificates

Given a validated incumbent with k components, CherryPicker tries to prove that no forest with at most $k - 1$ components exists. If it cannot prove this within its budget, it intentionally times out rather than printing an uncertified many-tree incumbent.

Direct-pair SAT.

For smaller instances the solver creates variables x_{ab} indicating that two leaves are in the same component. Transitivity clauses enforce an equivalence relation. Conflicting rooted triples are forbidden from lying in one component, edge clauses forbid component overlap in any input tree, and a sequential counter bounds the number of representatives by K . A valid K -component forest gives a satisfying assignment, so UNSAT proves target K impossible.

Reference-cut SAT.

The solver also encodes cut edges in the first input tree. A forest with at most K components corresponds to at most $K - 1$ reference cuts. Clauses force every conflicting triple to be separated by a selected cut. SAT models are decoded and validated; invalid decoded partitions are repaired or blocked by lazy clauses. Exhaustive UNSAT proves that no valid K -component forest exists.

Small-savings packing.

For targets near the singleton forest, write $s = n - K$. The solver enumerates valid components that can contribute to the required savings, stores leaf and edge masks, and solves an exact packing problem. A component C contributes $|C| - 1$ savings, so a disjoint packing of savings at least s is equivalent to a forest with at most K components. Exhaustive failure proves target K impossible.

► **Theorem 2.** *Every many-tree output printed by the certified CherryPicker core is optimal.*

Proof. The output forest is first validated, hence $OPT \leq k$ for its component count k . The solver prints only after one of the exact routines proves that no forest with at most $k - 1$ components exists, hence $OPT \geq k$. Therefore $OPT = k$. ◀

4 Two-Tree Route and Implementation

For $t = 2$ and $n \leq 500$, the wrapper dispatches to a bundled two-tree route based on the RS/Whidden code path [1] and an implementation-specific component-pool repair module layer. It constructs an incumbent, enriches a component pool using local edits and region generators, and solves set-packing subproblems with the statically linked HiGHS library. The final forest is repaired and validated before output. For $t = 2, n > 500$, CherryPicker intentionally times out.

The submitted executable is `solver_submit`. The wrapper reads the PACE `#p t n` header and dispatches to the embedded two-tree or many-tree solver. CaDiCaL is vendored for SAT certificates and HiGHS for integer programming. Trees are stored in compact arrays with LCA support, and the output is one Newick component per line.

References

- 1 C. Whidden, R. G. Beiko, and N. Zeh. Fixed-parameter algorithms for maximum agreement forests. *SIAM Journal on Computing*, 42(4):1431–1466, 2013.
- 2 Z. Li and N. Zeh. Computing maximum agreement forests without cluster partitioning is folly. In *25th Annual European Symposium on Algorithms (ESA 2017)*, LIPIcs 87, pages 56:1–56:14, 2017.
- 3 S. Kelk. A pedagogical implementation of the cluster reduction. <https://skelk.sdf-eu.org/cluster/>, accessed 2026-07-09.
- 4 A. Biere, K. Fazekas, M. Fleury, and M. Heisinger. CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020. In *Proc. SAT Competition 2020*, 2020.
- 5 Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10:119–142, 2018.